

# Le modèle des Transformers pour l'analyse d'image

Anrezki Ayoub

## Résumé

Les Transformers, introduits initialement pour le traitement du langage, sont depuis devenus une architecture centrale en analyse d'image, à travers notamment le Vision Transformer. Ce papier propose une présentation rigoureuse de cette architecture, destinée à un lecteur formé aux mathématiques mais étranger au domaine, où chaque objet est introduit par le besoin précis auquel il répond plutôt que posé sans justification. Nous partons du cadre statistique de l'apprentissage automatique pour motiver l'apparition du mécanisme d'attention, avant de construire l'architecture Transformer dans son intégralité (attention, encodage positionnel, architecture complète), puis son adaptation à l'analyse d'image. Tout au long du texte, nous distinguons explicitement trois régimes : les faits calculatoires, vérifiables directement ; les théorèmes établis dans la littérature, que nous prouvons ou admettons honnêtement avec référence selon leur portée ; et les choix d'architecture motivés par l'observation empirique plutôt que par une démonstration, présentés comme tels. Une courte excursion vers les grands modèles de langage complète le tableau.

## Table des matières

|          |                                                                       |          |
|----------|-----------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                   | <b>2</b> |
| <b>2</b> | <b>Préliminaires</b>                                                  | <b>2</b> |
| 2.1      | Le problème de la règle inconnue . . . . .                            | 2        |
| 2.2      | Le cadre statistique de l'apprentissage . . . . .                     | 3        |
| 2.2.1    | Représenter les entrées et les sorties . . . . .                      | 3        |
| 2.2.2    | Pourquoi une loi de probabilité plutôt qu'une fonction . . . . .      | 4        |
| 2.2.3    | Espace des hypothèses, perte, risque . . . . .                        | 4        |
| 2.3      | Optimisation . . . . .                                                | 5        |
| 2.3.1    | Le problème que l'optimisation doit résoudre . . . . .                | 5        |
| 2.3.2    | La descente de gradient . . . . .                                     | 6        |
| 2.3.3    | Pourquoi la descente stochastique (SGD) . . . . .                     | 6        |
| 2.3.4    | Calculer $\nabla_{\theta}\mathcal{L}$ : la rétropropagation . . . . . | 6        |
| 2.4      | Réseaux de neurones feedforward . . . . .                             | 7        |
| 2.4.1    | Pourquoi cette famille précisément . . . . .                          | 7        |
| 2.4.2    | Définition du réseau feedforward . . . . .                            | 7        |
| 2.4.3    | Le théorème d'approximation universelle . . . . .                     | 7        |
| 2.5      | Représentations vectorielles (embeddings) . . . . .                   | 10       |
| 2.6      | Vers les données séquentielles . . . . .                              | 10       |
| 2.6.1    | Mots et phrases : une définition inductive . . . . .                  | 10       |
| 2.6.2    | Le problème que pose une séquence . . . . .                           | 11       |
| 2.6.3    | Exemples de RNN . . . . .                                             | 11       |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 2.6.4    | Le problème du gradient qui s'évanouit ou explose . . . . .     | 12        |
| 2.6.5    | Une seconde limite, indépendante du gradient . . . . .          | 13        |
| <b>3</b> | <b>Attention et architecture Transformer</b>                    | <b>13</b> |
| 3.1      | Motivation . . . . .                                            | 13        |
| 3.2      | Le mécanisme d'attention . . . . .                              | 14        |
| 3.2.1    | Projections . . . . .                                           | 14        |
| 3.2.2    | La fonction softmax . . . . .                                   | 14        |
| 3.2.3    | Définition de l'attention . . . . .                             | 14        |
| 3.2.4    | Exemple numérique . . . . .                                     | 15        |
| 3.2.5    | Justification de la mise à l'échelle par $\sqrt{d_k}$ . . . . . | 15        |
| 3.2.6    | Le lemme du barycentre . . . . .                                | 16        |
| 3.3      | Multi-head attention . . . . .                                  | 16        |
| 3.3.1    | Motivation . . . . .                                            | 16        |
| 3.3.2    | Définition . . . . .                                            | 17        |
| 3.3.3    | Un point à ne pas sur-interpréter . . . . .                     | 17        |
| 3.4      | Nécessité d'un encodage positionnel . . . . .                   | 17        |
| 3.4.1    | Un théorème d'équivariance . . . . .                            | 17        |
| 3.4.2    | L'encodage positionnel sinusoïdal . . . . .                     | 18        |
| 3.5      | Architecture complète . . . . .                                 | 19        |
| 3.5.1    | Connexions résiduelles . . . . .                                | 19        |
| 3.5.2    | Normalisation . . . . .                                         | 19        |
| 3.5.3    | Réseau feedforward position-wise . . . . .                      | 20        |
| 3.5.4    | Un bloc complet . . . . .                                       | 20        |

## 1 Introduction

Les Transformers, introduits par Vaswani et al. (2017), sont aujourd'hui l'architecture dominante en traitement du langage et, depuis les Vision Transformers, en analyse d'image. Pourtant, la plupart des présentations disponibles admettent l'architecture plus qu'elles ne la démontrent : les objets mathématiques sous-jacents (produit scalaire, softmax, espaces de représentation) sont rarement posés avec la rigueur qu'un lecteur habitué à la preuve est en droit d'attendre.

Ce papier tente de combler cet écart, en distinguant à chaque étape ce qui est un fait démontrable, ce qui est un théorème établi ailleurs et admis avec référence, et ce qui n'est qu'un choix d'architecture motivé empiriquement.

## 2 Préliminaires

### 2.1 Le problème de la règle inconnue

Il existe des tâches pour lesquelles on connaît une suite finie d'instructions qui, à partir d'une entrée, produit la sortie voulue, et dont on peut prouver la correction : trier une liste, tester la primalité d'un entier, ou encore calculer la dérivée d'une expression symbolique donnée sous forme d'un arbre syntaxique explicite (une composition d'opérations élémentaires). Dans ce dernier cas, la dérivation formelle procède par récurrence structurelle sur l'arbre (linéarité pour une somme, règle du produit, règle de la chaîne pour une composition, table pour les fonctions

atomiques), et sa correction se prouve par induction structurelle. Aucun apprentissage n'est nécessaire : la règle est connue et prouvée.

Il existe d'autres tâches pour lesquelles personne n'a jamais écrit un tel algorithme, alors même que des humains les résolvent couramment : reconnaître qu'une image montre un chat, juger de la grammaticalité d'une phrase. Ce ne sont pas des tâches mal posées : on sait reconnaître une bonne réponse d'une mauvaise. Ce qui manque, c'est une façon d'écrire la règle. Personne ne peut décrire, en une suite finie d'instructions portant sur des intensités de pixels, ce qui distingue un chat d'un chien.

### Nota

Le critère qui sépare ces deux catégories n'est pas « facile » contre « difficile », mais l'existence ou non d'une règle explicite dont on puisse prouver la correction. Il vaut la peine de noter, en anticipant sur la suite, qu'il est possible d'entraîner un système par apprentissage à deviner la dérivée d'une formule symbolique (traitée comme une tâche de traduction : une formule en entrée, une formule en sortie), avec une précision élevée sur des formules jamais vues, alors même que l'algorithme exact existe déjà pour ce problème. Ce fait, sur lequel on reviendra, montre que l'apprentissage n'est pas réservé aux problèmes sans solution connue : il devient seulement *facultatif* dès qu'une règle explicite existe, et *nécessaire* quand aucune n'est connue.

Le changement de paradigme consiste alors à ne plus écrire la règle soi-même, mais à la faire produire par une machine à partir d'exemples : on donne des paires (entrée, sortie correcte), par exemple des images déjà étiquetées « chat » ou « chien », et on demande à un algorithme de produire, à partir de ces exemples, une règle qui fonctionne aussi sur des entrées jamais vues. Chaque objet introduit dans ce qui suit répond à un besoin précis de ce programme.

## 2.2 Le cadre statistique de l'apprentissage

### 2.2.1 Représenter les entrées et les sorties

On a vu en 2.1 qu'il s'agit, étant donnée une entrée, de produire la sortie correcte associée, sans disposer d'une règle explicite pour le faire. On formalise à présent les ensembles dans lesquels vivent ces objets.

### Définition (Problème de prédiction)

Un *problème de prédiction* est la donnée d'un couple d'ensembles non vides  $(\mathcal{X}, \mathcal{Y})$ . On appelle  $\mathcal{X}$  l'**espace des entrées** du problème et  $\mathcal{Y}$  son **espace des sorties** : les entrées possibles pour ce problème sont, par définition, les éléments de  $\mathcal{X}$ , et les sorties possibles les éléments de  $\mathcal{Y}$ . Ce qui distingue, pour une entrée  $x \in \mathcal{X}$  donnée, une sortie  $y \in \mathcal{Y}$  correcte d'une sortie incorrecte n'est pas encore précisé à ce stade ; ce sera l'objet de 2.2.2.

Un problème de prédiction  $(\mathcal{X}, \mathcal{Y})$  est dit de **classification** si  $\mathcal{Y}$  est fini (ses éléments étant alors identifiés, sans perte de généralité, à  $\{1, \dots, K\}$  où  $K = |\mathcal{Y}|$ ), et de **régression** si  $\mathcal{Y} = \mathbb{R}$  (ou  $\mathbb{R}^m$ ).

**Exemple.** « Chat ou chien » est un problème de classification avec  $K = 2$ , où  $\mathcal{X} = [0, 1]^{H \times W \times 3}$  pour une résolution d'image  $H \times W$  à trois canaux de couleur fixée une fois pour toutes. Prédire

une température est un problème de régression,  $\mathcal{Y} = \mathbb{R}$ .

### 2.2.2 Pourquoi une loi de probabilité plutôt qu'une fonction

#### Nota

On dira qu'une relation entre  $\mathcal{X}$  et  $\mathcal{Y}$  est *fonctionnelle* si à chaque  $x$  correspond un unique  $y$ , c'est-à-dire si c'est le graphe d'une fonction  $f : \mathcal{X} \rightarrow \mathcal{Y}$ .

Deux faits empêchent la relation « entrée, bonne étiquette » d'être fonctionnelle en général : l'ambiguïté intrinsèque (deux occurrences identiques de  $x$  annotées différemment selon le contexte) et le bruit d'observation (erreur d'annotation). On remplace donc l'hypothèse d'une fonction  $f$  par celle, plus faible, d'une loi jointe  $\mathcal{D}$  sur  $\mathcal{X} \times \mathcal{Y}$ , inconnue, fixée, jamais observée directement, seulement échantillonnée :  $(X, Y) \sim \mathcal{D}$ . Si  $\mathcal{D}$  était connue, il n'y aurait rien à apprendre.

### 2.2.3 Espace des hypothèses, perte, risque

On cherche une fonction  $h : \mathcal{X} \rightarrow \mathcal{Y}$  qui approche au mieux le comportement de  $\mathcal{D}$ . Deux objets sont nécessaires pour rendre cette recherche précise :

- un **espace des hypothèses**  $\mathcal{H}$ , l'ensemble des fonctions  $h$  parmi lesquelles on cherche (pour les réseaux de neurones,  $\mathcal{H}$  sera l'ensemble des fonctions représentables par une architecture fixée, paramétrée par ses poids) ;
- une **fonction de perte**  $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$ , qui mesure l'écart entre une prédiction et la vraie valeur (perte 0-1,  $\ell(\hat{y}, y) = \mathbb{I}[\hat{y} \neq y]$ , pour la classification ; perte quadratique,  $\ell(\hat{y}, y) = (\hat{y} - y)^2$ , pour la régression).

#### Définition (Risque)

Le risque de  $h \in \mathcal{H}$  est

$$R(h) = \mathbb{E}_{(X,Y) \sim \mathcal{D}}[\ell(h(X), Y)].$$

C'est la quantité qu'on veut minimiser, mais elle dépend de  $\mathcal{D}$ , donc elle n'est pas calculable.

Ce qu'on observe est un échantillon  $S = ((x_1, y_1), \dots, (x_n, y_n))$  tiré i.i.d. selon  $\mathcal{D}$ .

#### Définition (Risque empirique)

$$\hat{R}_S(h) = \frac{1}{n} \sum_{i=1}^n \ell(h(x_i), y_i).$$

#### Convention (Principe ERM)

On choisit  $\hat{h} \in \mathcal{H}$  minimisant  $\hat{R}_S$  sur  $\mathcal{H}$ , au sens précisé par la définition 2.3.1 ci-dessous.

**La question centrale**, qu'il faut poser avant de continuer : pourquoi minimiser  $\hat{R}_S(h)$  devrait-il dire quelque chose sur  $R(h)$  ? Rien ne l'exclut *a priori* : on peut avoir  $\hat{R}_S(h) = 0$  avec  $R(h)$

arbitrairement mauvais (un  $h$  qui apprend l'échantillon par cœur). L'écart  $R(h) - \hat{R}_S(h)$  et son contrôle (loi des grands nombres pour  $h$  fixé, VC-dimension ou complexité de Rademacher pour  $h$  choisi après coup) sont l'objet de la théorie statistique de l'apprentissage. Nous laissons ce contrôle de côté pour l'instant, et renvoyons le lecteur à [5] pour un traitement complet ; il n'est pas nécessaire à la compréhension de ce qui suit.

## 2.3 Optimisation

### 2.3.1 Le problème que l'optimisation doit résoudre

Le principe ERM ramène l'apprentissage à un problème de minimisation : trouver  $\hat{h} \in \mathcal{H}$  qui minimise  $\hat{R}_S(h)$ . Pour que ce problème soit traitable algorithmiquement, il faut que  $\mathcal{H}$  soit paramétré de façon exploitable. On suppose désormais que  $\mathcal{H} = \{h_\theta : \theta \in \Theta\}$ , où  $\Theta \subseteq \mathbb{R}^p$  est un sous-espace de paramètres de dimension finie  $p$ , et  $\theta \mapsto h_\theta$  une famille de fonctions fixée (l'architecture). Minimiser  $\hat{R}_S(h)$  sur  $\mathcal{H}$  revient alors à minimiser

$$\mathcal{L}(\theta) := \hat{R}_S(h_\theta) = \frac{1}{n} \sum_{i=1}^n \ell(h_\theta(x_i), y_i)$$

sur  $\Theta$ , un problème d'optimisation en dimension finie, mais avec  $p$  potentiellement immense (des millions, voire des milliards de paramètres pour un Transformer), ce qui exclut d'emblée les méthodes qui nécessitent d'inverser une matrice de taille  $p \times p$  (comme la méthode de Newton) ou d'énumérer  $\Theta$ .

#### Définition (Minimiser)

Soit  $f : \Theta \rightarrow \mathbb{R}$  et  $\Theta \neq \emptyset$ . On dit que  $\theta^* \in \Theta$  *minimise*  $f$  sur  $\Theta$  si  $f(\theta^*) \leq f(\theta)$  pour tout  $\theta \in \Theta$ . On note alors  $\theta^* \in \arg \min_{\theta \in \Theta} f(\theta)$ .

#### Remarque

Rien ne garantit qu'un tel  $\theta^*$  existe. Ce que garantit toujours l'analyse, c'est l'existence de la borne inférieure  $\inf_{\theta \in \Theta} f(\theta) \in \mathbb{R} \cup \{-\infty\}$  (un ensemble non vide de réels minorés admet toujours une borne inférieure, par la propriété de la borne inférieure de  $\mathbb{R}$ ), mais cette borne peut ne jamais être atteinte par aucun point de  $\Theta$  (exemple simple :  $f(\theta) = e^{-\theta}$  sur  $\Theta = \mathbb{R}$ ,  $\inf f = 0$ , jamais atteint). L'existence effective d'un minimiseur est garantie, par exemple, si  $\Theta$  est compact et  $f$  continue, par le **théorème des bornes atteintes** (théorème de Weierstrass) :  $f$  continue sur un compact  $K$  atteint son minimum et son maximum sur  $K$ . Ce n'est en général **pas** le cas ici :  $\Theta = \mathbb{R}^p$  n'est pas compact.

*Convention adoptée pour la suite* : quand on écrit «  $\hat{h}$  minimise  $\hat{R}_S(h)$  sur  $\mathcal{H}$  », il faut comprendre cette expression au sens faible suivant : on ne cherche pas un point qui atteint exactement l'infimum, mais un point  $\theta$  tel que  $\mathcal{L}(\theta)$  soit arbitrairement proche de  $\inf_{\Theta} \mathcal{L}$ , ce que produit précisément un algorithme itératif comme la descente de gradient (sans garantie que cette limite soit l'infimum global si  $\mathcal{L}$  n'est pas convexe).

### 2.3.2 La descente de gradient

**Idée.** Si  $\mathcal{L}$  est différentiable,  $-\nabla\mathcal{L}(\theta)$  indique, localement, la direction de plus forte décroissance de  $\mathcal{L}$  en  $\theta$ . On construit donc une suite  $(\theta_t)_{t \geq 0}$  par

$$\theta_{t+1} = \theta_t - \eta \nabla\mathcal{L}(\theta_t),$$

où  $\eta > 0$  est le pas d'apprentissage (*learning rate*).

#### Proposition (Décroissance locale)

Pour  $\eta$  suffisamment petit,  $\mathcal{L}(\theta_{t+1}) \leq \mathcal{L}(\theta_t)$ , avec égalité seulement si  $\nabla\mathcal{L}(\theta_t) = 0$ .

*Preuve.* Développement de Taylor à l'ordre 1 :

$$\mathcal{L}(\theta_t - \eta \nabla\mathcal{L}(\theta_t)) = \mathcal{L}(\theta_t) - \eta \|\nabla\mathcal{L}(\theta_t)\|^2 + o(\eta),$$

donc pour  $\eta$  petit le terme dominant est  $-\eta \|\nabla\mathcal{L}(\theta_t)\|^2 \leq 0$ . □

#### Remarque

Ceci garantit une décroissance *locale*, pas la convergence vers un minimum global.  $\mathcal{L}$ , construite à partir d'un réseau de neurones, est en général non convexe : la descente de gradient peut converger vers un minimum local, un point-selle, ou un plateau. C'est un fait empirique documenté que ce n'est en général pas rédhibitoire en pratique pour les grands réseaux [3], mais aucune garantie théorique de proximité au minimum global n'existe dans ce cadre général.

### 2.3.3 Pourquoi la descente stochastique (SGD)

Calculer  $\nabla\mathcal{L}(\theta_t)$  exactement demande de parcourir les  $n$  termes de la somme  $\hat{R}_S$ , ce qui devient coûteux quand  $n$  est grand. On observe que  $\nabla\mathcal{L}(\theta) = \frac{1}{n} \sum_i \nabla_{\theta} \ell(h_{\theta}(x_i), y_i)$  est lui-même une moyenne : on peut donc l'estimer sans biais à partir d'un sous-ensemble  $B \subset \{1, \dots, n\}$  (un *mini-batch*) tiré aléatoirement, en calculant  $\frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell(h_{\theta}(x_i), y_i)$ .

#### Proposition (Estimateur sans biais)

$$\mathbb{E}_B \left[ \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell(h_{\theta}(x_i), y_i) \right] = \nabla\mathcal{L}(\theta).$$

*Preuve.* Par linéarité de l'espérance et le fait que chaque indice a la même probabilité d'être tiré. □

### 2.3.4 Calculer $\nabla_{\theta}\mathcal{L}$ : la rétropropagation

Reste le problème calculatoire :  $h_{\theta}$  pour un réseau de neurones est une composition de nombreuses fonctions élémentaires (couches), chacune paramétrée par une partie de  $\theta$ . Calculer  $\nabla_{\theta}\mathcal{L}$  à la main, terme par terme, serait ingérable dès que le nombre de couches grandit.

Le fait mathématique unique qui résout tout est la règle de la chaîne. Si  $h_\theta = f_L \circ f_{L-1} \circ \dots \circ f_1$ , où chaque  $f_k$  dépend d'un sous-vecteur de paramètres  $\theta_k$ , alors la dérivée de  $\mathcal{L}$  par rapport à  $\theta_k$  se calcule en propageant les dérivées de la sortie vers l'entrée, couche par couche, chaque couche ne calculant que la dérivée locale de sa propre fonction. C'est ce qu'on appelle la **rétropropagation** [7] : ce n'est pas un algorithme d'apprentissage à part entière, seulement une organisation efficace (en temps linéaire en le nombre de couches, par un unique parcours arrière du graphe de calcul) du calcul de la règle de la chaîne sur une composition profonde.

## 2.4 Réseaux de neurones feedforward

### 2.4.1 Pourquoi cette famille précisément

Le principe ERM (2.2.3) suppose donné un espace des hypothèses  $\mathcal{H}$ , mais ne dit rien sur sa forme. Pour que la descente de gradient (2.3) s'applique, il faut que  $\mathcal{H} = \{h_\theta : \theta \in \Theta\}$  soit une famille de fonctions différentiables en  $\theta$ . Mais différentiabilité ne suffit pas : il faut aussi que  $\mathcal{H}$  soit assez riche pour contenir (ou approcher) la fonction qu'on cherche, quelle qu'elle soit. Une famille trop pauvre (par exemple, les fonctions affines) ne pourra jamais représenter une relation entrée-sortie complexe, aussi bien optimisée soit-elle.

#### Remarque

Une composition de fonctions affines est affine : si  $f_1(x) = A_1x + b_1$  et  $f_2(x) = A_2x + b_2$ , alors  $f_2(f_1(x)) = A_2A_1x + A_2b_1 + b_2$ , encore affine. Donc empiler des couches purement linéaires, aussi nombreuses soient-elles, ne donne jamais plus de pouvoir expressif qu'une seule couche. C'est cette observation, et elle seule, qui impose l'ajout d'une **fonction d'activation** non linéaire entre les couches.

### 2.4.2 Définition du réseau feedforward

#### Définition (Perceptron multicouche)

Une *couche* de taille  $(d_{\text{in}}, d_{\text{out}})$  est la donnée d'une matrice de poids  $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ , d'un biais  $b \in \mathbb{R}^{d_{\text{out}}}$ , et d'une fonction d'activation  $\sigma : \mathbb{R} \rightarrow \mathbb{R}$  appliquée composante par composante, définissant  $f(x) = \sigma(Wx + b)$ . Un *réseau feedforward* (ou perceptron multicouche, MLP) à  $L$  couches est la composition  $h_\theta = f_L \circ \dots \circ f_1$ , où  $\theta$  rassemble tous les  $(W_i, b_i)$ .

### 2.4.3 Le théorème d'approximation universelle

On voudrait justifier que cette famille est un choix légitime pour  $\mathcal{H}$ , au sens où elle peut approcher n'importe quelle fonction continue raisonnable. C'est le **théorème d'approximation universelle** [2]. Nous en donnons une preuve complète, qui s'appuie sur quatre outils externes d'analyse fonctionnelle et de théorie de la mesure, standards et admis sans démonstration : le théorème de Hahn-Banach, le théorème de représentation de Riesz pour  $C(K)$ , le théorème de convergence dominée, et l'injectivité de la transformée de Fourier des mesures signées finies [1, 6]. Les reprouver serait un détour disproportionné par rapport au sujet de cet article.

**Cadre.** On travaille sur  $I_n = [0, 1]^n$ , muni de  $C(I_n)$ , l'espace des fonctions continues  $I_n \rightarrow \mathbb{R}$ , normé par  $\|f\|_\infty = \sup_{x \in I_n} |f(x)|$ .

### Définition (Fonction sigmoïdale)

$\sigma : \mathbb{R} \rightarrow \mathbb{R}$  est *sigmoïdale* si  $\sigma(t) \rightarrow 1$  quand  $t \rightarrow +\infty$  et  $\sigma(t) \rightarrow 0$  quand  $t \rightarrow -\infty$ .

### Nota

Une fonction continue sigmoïdale est nécessairement bornée : par continuité elle est bornée sur un compact  $[-T, T]$  assez grand pour que  $|\sigma(t) - 1| < 1$  pour  $t > T$  et  $|\sigma(t)| < 1$  pour  $t < -T$ ; ces trois bornes combinées bornent  $\sigma$  sur  $\mathbb{R}$  tout entier.

### Définition (Espace $S_\sigma$ )

Pour  $\sigma$  fixée,

$$S_\sigma := \left\{ \sum_{j=1}^N \alpha_j \sigma(w_j^T x + b_j) : N \in \mathbb{N}, w_j \in \mathbb{R}^n, \alpha_j, b_j \in \mathbb{R} \right\} \subset C(I_n),$$

l'ensemble des fonctions calculables par un réseau à une couche cachée d'activation  $\sigma$ , de largeur arbitraire, et une sortie linéaire.  $S_\sigma$  est un sous-espace vectoriel de  $C(I_n)$  (trivial : combiner deux sommes finies de cette forme, ou en multiplier une par un scalaire, donne encore une somme de cette forme).

### Définition (Fonction discriminante)

$\sigma$  bornée mesurable est *discriminante* (relativement à  $I_n$ ) si, pour toute mesure signée régulière finie  $\mu$  sur  $I_n$ ,

$$\left( \forall w \in \mathbb{R}^n, \forall b \in \mathbb{R}, \int_{I_n} \sigma(w^T x + b) d\mu(x) = 0 \right) \implies \mu = 0.$$

**Bloc 1 : discriminante  $\Rightarrow$  dense.**

### Lemme (A)

Si  $\sigma$  est discriminante,  $S_\sigma$  est dense dans  $C(I_n)$ .

*Preuve.* Soit  $\bar{S}$  l'adhérence de  $S_\sigma$  dans  $(C(I_n), \|\cdot\|_\infty)$ , un sous-espace fermé. Supposons par l'absurde  $\bar{S} \neq C(I_n)$ . Par le théorème de Hahn-Banach (corollaire de séparation, admis [1]), il existe une forme linéaire continue non nulle  $L \in C(I_n)^*$  qui s'annule sur  $\bar{S}$ .

Par le théorème de représentation de Riesz pour  $C(K)$ ,  $K$  compact métrique (admis [6]),  $C(I_n)^* \cong M(I_n)$ , l'espace des mesures signées régulières finies sur  $I_n$  : il existe une unique  $\mu \in M(I_n)$  telle que  $L(h) = \int_{I_n} h d\mu$  pour tout  $h$ , et  $\mu \neq 0$  puisque  $L \neq 0$ .

Pour tous  $w, b$ , la fonction  $x \mapsto \sigma(w^T x + b)$  appartient à  $S_\sigma \subseteq \bar{S}$  (cas  $N = 1, \alpha_1 = 1$ ), donc  $L$  s'y annule :

$$\int_{I_n} \sigma(w^T x + b) d\mu(x) = 0 \quad \text{pour tous } w, b.$$

Comme  $\sigma$  est discriminante,  $\mu = 0$  : contradiction. Donc  $\bar{S} = C(I_n)$ . □



**Bloc 2 : toute sigmoïdale continue est discriminante.**

**Lemme (B)**

Si  $\sigma$  est continue et sigmoïdale, elle est discriminante.

*Preuve.* Soit  $\mu$  une mesure signée régulière finie sur  $I_n$  telle que  $\int \sigma(w^T x + b) d\mu(x) = 0$  pour tous  $w, b$ . Montrons  $\mu = 0$ .

*Étape 1 (passage à la limite sur une droite affine).* Fixons  $w \in \mathbb{R}^n, b, c \in \mathbb{R}$ . Pour  $\lambda > 0$ ,  $\sigma_\lambda(x) := \sigma(\lambda(w^T x + b) + c)$  est de la forme  $\sigma(w_\lambda^T x + b_\lambda)$  avec  $w_\lambda = \lambda w, b_\lambda = \lambda b + c$ , donc par hypothèse

$$\int_{I_n} \sigma_\lambda d\mu = 0 \quad \text{pour tout } \lambda > 0. \quad (*)$$

Quand  $\lambda \rightarrow +\infty$ , pour  $x$  fixé : si  $w^T x + b > 0$ ,  $\sigma_\lambda(x) \rightarrow 1$ ; si  $w^T x + b < 0$ ,  $\sigma_\lambda(x) \rightarrow 0$ ; si  $w^T x + b = 0$ ,  $\sigma_\lambda(x) = \sigma(c)$  constamment. Donc  $\sigma_\lambda \rightarrow \gamma$  ponctuellement, où  $\gamma = \mathbb{1}_{H^+} + \sigma(c)\mathbb{1}_{\partial H}$ , avec  $H^+ = \{w^T x + b > 0\}$ ,  $\partial H = \{w^T x + b = 0\}$  (intersectés avec  $I_n$ ). Comme  $\sigma$  est bornée (Nota ci-dessus),  $(\sigma_\lambda)$  est uniformément bornée; le théorème de convergence dominée (admis, standard) donne  $\int \sigma_\lambda d\mu \rightarrow \int \gamma d\mu = \mu(H^+) + \sigma(c)\mu(\partial H)$ . Avec  $(*)$  :

$$\mu(H^+) + \sigma(c)\mu(\partial H) = 0 \quad \text{pour tout } c \in \mathbb{R}. \quad (**)$$

*Étape 2 (élimination de  $c$ ).* Faisant  $c \rightarrow +\infty$  dans  $(**)$  ( $\sigma(c) \rightarrow 1$ ) :  $\mu(H^+) + \mu(\partial H) = 0$ . Faisant  $c \rightarrow -\infty$  ( $\sigma(c) \rightarrow 0$ ) :  $\mu(H^+) = 0$ . D'où aussi  $\mu(\partial H) = 0$ . Comme  $w, b$  étaient arbitraires :

$$\mu(\{x \in I_n : w^T x \geq t\}) = 0 \quad \text{pour tous } w \in \mathbb{R}^n, t \in \mathbb{R}. \quad (\dagger)$$

*Étape 3 (mesure image et unicité).* Fixons  $w$ , et soit  $\nu_w := \mu \circ \pi_w^{-1}$  la mesure image sur  $\mathbb{R}$  par  $\pi_w(x) = w^T x$  (mesure signée finie bien définie,  $\pi_w$  continue donc borélienne,  $I_n$  compact). Par  $(\dagger)$ ,  $\nu_w([t, \infty)) = 0$  pour tout  $t \in \mathbb{R}$ . Les demi-droites  $[t, \infty)$  forment un  $\pi$ -système engendrant la tribu borélienne de  $\mathbb{R}$ ; une mesure signée finie qui s'annule sur un tel système générateur est nulle (fait standard, admis; c'est exactement l'unicité d'une mesure via sa fonction de répartition). Donc  $\nu_w \equiv 0$ .

*Étape 4 (transformée de Fourier).* Pour toute  $f$  continue bornée,  $\int_{I_n} f(w^T x) d\mu(x) = \int_{\mathbb{R}} f d\nu_w = 0$ . En particulier, pour  $f(t) = \cos t$  et  $f(t) = \sin t$ , et comme  $w$  parcourt  $\mathbb{R}^n$  tout entier :

$$\hat{\mu}(w) := \int_{I_n} e^{iw^T x} d\mu(x) = 0 \quad \text{pour tout } w \in \mathbb{R}^n.$$

La transformée de Fourier de  $\mu$  (étendue par 0 hors de  $I_n$ ) est identiquement nulle. Par injectivité de la transformée de Fourier sur les mesures signées finies (admis [6]),  $\mu = 0$ .  $\square$

**Théorème (Approximation universelle, Cybenko 1989)**

Si  $\sigma$  est continue et sigmoïdale, alors  $S_\sigma$  est dense dans  $C(I_n)$  : pour tout  $f \in C(I_n)$  et tout  $\varepsilon > 0$ , il existe  $N, w_j, \alpha_j, b_j$  tels que  $|\sum_j \alpha_j \sigma(w_j^T x + b_j) - f(x)| < \varepsilon$  pour tout  $x \in I_n$ .

*Preuve.* Le lemme B donne que  $\sigma$  est discriminante; le lemme A conclut.  $\square$

## 2.5 Représentations vectorielles (embeddings)

Tout ce qu'on a construit jusqu'ici ( $h_\theta$  un MLP, calculé par produits matriciels et activations, 2.4) suppose que l'entrée est déjà un vecteur réel. Mais beaucoup de problèmes de prédiction (2.2.1) ont un espace des entrées  $\mathcal{X}$  qui n'est pas naturellement un sous-ensemble de  $\mathbb{R}^d$  : c'est un ensemble discret, par exemple un ensemble fini de catégories ou de symboles. Il faut donc un objet qui transforme un élément discret en un vecteur, avant que quoi que ce soit d'autre ne puisse s'appliquer.

### Définition (Plongement)

Soit  $V$  un ensemble fini non vide. Un *plongement* (ou *embedding*) est une application  $e : V \rightarrow \mathbb{R}^d$ . En pratique,  $e$  est représenté par une matrice  $E \in \mathbb{R}^{|V| \times d}$ , où la ligne indexée par  $v \in V$  est  $e(v)$  : rechercher  $e(v)$  revient à extraire une ligne de  $E$ .

### Remarque

Rien dans cette définition ne garantit que  $e$  capture une quelconque notion de similarité sémantique (que deux éléments proches en sens aient des vecteurs proches en norme). C'est un fait empirique observé une fois  $E$  entraînée par descente de gradient comme n'importe quel autre paramètre de  $\theta$  (2.3), pas une propriété qui découle de la définition elle-même. On se gardera, plus loin dans l'article, de présenter cette propriété comme démontrée.

## 2.6 Vers les données séquentielles

### 2.6.1 Mots et phrases : une définition inductive

#### Définition (Mot)

Soit  $V$  un ensemble fini non vide, qu'on appelle *vocabulaire* ; ses éléments sont appelés des *mots*. Aucune structure interne n'est supposée sur  $V$  : un mot est un atome.

#### Définition (Phrase, inductive)

L'ensemble  $V^*$  des *phrases* sur  $V$  est défini inductivement par :

- (cas de base) la séquence vide  $\varepsilon$  est une phrase ;
- (cas inductif) si  $w \in V$  est un mot et  $p \in V^*$  est une phrase, alors  $w \cdot p$  (la concaténation de  $w$  à gauche de  $p$ ) est une phrase.

#### Nota

On retrouve ici exactement le principe de construction déjà utilisé pour les termes (un ensemble d'atomes, une opération qui construit un objet composé à partir d'objets déjà construits), et la même notation  $V^*$  que pour les séquences de positions. La *longueur*  $|p| \in \mathbb{N}$  d'une phrase se définit alors, sans surprise, par récurrence structurale :  $|\varepsilon| = 0$ ,  $|w \cdot p| = 1 + |p|$ .

### Remarque

Ce qu'on appelle ici « mot » n'a rien de linguistique : c'est un élément arbitraire d'un ensemble fini fixé une fois pour toutes. En pratique,  $V$  est construit par un algorithme de tokenisation (le plus courant étant le *byte-pair encoding*) qui découpe le texte en unités pas nécessairement alignées sur les mots au sens usuel (un mot rare peut être coupé en plusieurs *tokens*). Nous ne détaillons pas ce mécanisme, extérieur au sujet mathématique de cet article ; on garde le terme « mot » par commodité, en gardant à l'esprit qu'il désigne en réalité un token.

Ceci donne, pour un problème de prédiction (2.2.1) portant sur du texte,  $\mathcal{X} = V^*$  (ou  $V^{\leq T}$  si l'on borne la longueur), et chaque phrase  $p = w_1 \cdot w_2 \cdots w_T \in \mathcal{X}$  se transforme, via le plongement  $e$  de 2.5, en une séquence  $(x_1, \dots, x_T) = (e(w_1), \dots, e(w_T))$  de vecteurs de  $\mathbb{R}^d$ .

### 2.6.2 Le problème que pose une séquence

Une phrase, une fois transformée en  $(x_1, \dots, x_T)$  par le plongement, n'est pas un vecteur de taille fixe :  $T$  varie selon l'exemple. Un MLP (2.4), construit pour une entrée de dimension fixe, ne s'applique pas tel quel. Il faut une architecture qui, à la fois, accepte une longueur variable, et partage les mêmes paramètres à chaque position (sans quoi le nombre de paramètres croîtrait avec  $T$ , et rien n'obligerait le réseau à traiter « le troisième mot » de la même façon quelle que soit la phrase).

### Définition (Réseau de neurones récurrent, RNN)

Un RNN est donné par une fonction  $f_\theta : \mathbb{R}^{d_h} \times \mathbb{R}^d \rightarrow \mathbb{R}^{d_h}$ , appliquée récursivement :

$$h_0 = 0, \quad h_t = f_\theta(h_{t-1}, x_t) \quad \text{pour } t = 1, \dots, T,$$

où  $h_t$  est l'*état caché* au temps  $t$ . Le même  $\theta$  (les mêmes poids) est utilisé à chaque pas de temps : c'est le partage de paramètres qui rend l'architecture applicable à une séquence de longueur arbitraire.

### 2.6.3 Exemples de RNN

**Exemple (calcul explicite).** Prenons  $V = \{\text{« le »}, \text{« chat »}, \text{« dort »}\}$ , un plongement jouet  $e(\text{« le »}) = (1, 0)$ ,  $e(\text{« chat »}) = (0, 1)$ ,  $e(\text{« dort »}) = (1, 1)$ , et un RNN de dimension cachée  $d_h = 1$  avec  $f_\theta(h, x) = \tanh(Ah + Bx)$ ,  $A = 0.5$ ,  $B = (0.5, -0.5)$ . Sur la phrase  $p = \text{« le »} \cdot \text{« chat »} \cdot \text{« dort »}$  :

$$h_0 = 0, \quad h_1 = \tanh(0.5 \cdot 0 + 0.5 \cdot 1 - 0.5 \cdot 0) = \tanh(0.5) \approx 0.462,$$

$$h_2 = \tanh(0.5 \cdot 0.462 + 0.5 \cdot 0 - 0.5 \cdot 1) \approx \tanh(-0.269) \approx -0.263,$$

$$h_3 = \tanh(0.5 \cdot (-0.263) + 0.5 \cdot 1 - 0.5 \cdot 1) \approx \tanh(-0.131) \approx -0.131.$$

Cet exemple rend visible ce que la définition récursive cache :  $h_3$  dépend de toute la phrase, mais l'influence de  $x_1 = e(\text{« le »})$  sur  $h_3$  passe par deux applications de  $A$ , donc s'atténue déjà (facteur  $\approx A^2 = 0.25$  dans le cas linéarisé) : une préfiguration numérique de la Proposition du paragraphe suivant.

**Exemple (trois usages typiques).** Selon ce qu'on fait de la séquence  $(h_1, \dots, h_T)$  :

- *classification de séquence* : on ne garde que  $h_T$ , et on applique un MLP (2.4) suivi d'une perte de classification (2.2.3), par exemple l'analyse de sentiment, un problème de classification au sens de 2.2.1 ;
- *modélisation de langage* : à chaque  $t$ , on prédit le mot suivant  $w_{t+1}$  à partir de  $h_t$ , un problème de classification sur  $V$  tout entier ( $K = |V|$ ) ;
- *traduction* (séquence-à-séquence) : un premier RNN encode  $(x_1, \dots, x_T)$  en un unique vecteur  $h_T$ , un second RNN décode  $h_T$  en une nouvelle séquence. C'est précisément ce cas, où toute l'information de la phrase source doit transiter par le seul vecteur  $h_T$ , qui motivera directement le mécanisme d'attention en Partie 3 :  $h_T$  devient un goulot d'étranglement pour les phrases longues.

**Exemple numérique (évanouissement, cas linéaire).** Avec  $A = 0.5$  (scalaire,  $d_h = 1$ ) :  $A^{10} \approx 9.8 \times 10^{-4}$ ,  $A^{50} \approx 8.9 \times 10^{-16}$  : au-delà d'une cinquantaine de pas, le gradient relatif à  $h_0$  est numériquement indiscernable de zéro en simple précision flottante. Avec  $A = 1.5$  à l'inverse,  $A^{50} \approx 6.4 \times 10^8$  : explosion. Ces deux nombres, à eux seuls, disent pourquoi un pas de temps  $\eta$  fixe (2.3.2) ne peut pas s'accommoder des deux régimes à la fois.

#### 2.6.4 Le problème du gradient qui s'évanouit ou explose

On voudrait entraîner ce réseau par rétropropagation (2.3.4), donc calculer  $\partial \mathcal{L} / \partial h_0$ , ce qui demande de faire passer le gradient à travers les  $T$  applications successives de  $f_\theta$ . C'est là qu'apparaît un phénomène qu'on peut isoler précisément, dans un cas simplifié.

##### Proposition (Cas linéaire)

Soit  $f_\theta(h_{t-1}, x_t) = Ah_{t-1}$  (on retire volontairement la dépendance en  $x_t$  et toute non-linéarité, pour isoler le phénomène). Alors  $h_T = A^T h_0$ , et si  $A$  est diagonalisable avec valeurs propres  $\lambda_1, \dots, \lambda_{d_h}$ ,  $\|A^T\|$  croît exponentiellement en  $T$  si  $\max_i |\lambda_i| > 1$ , et décroît exponentiellement vers 0 si  $\max_i |\lambda_i| < 1$ .

*Preuve.* Écrivons  $A = P\Lambda P^{-1}$ ,  $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_{d_h})$ . Alors  $A^T = P\Lambda^T P^{-1}$ , avec  $\Lambda^T = \text{diag}(\lambda_1^T, \dots, \lambda_{d_h}^T)$ . Si  $\rho := \max_i |\lambda_i| > 1$ , la composante associée à l'indice réalisant ce maximum vérifie  $|\lambda_i^T| = \rho^T \rightarrow \infty$  ; comme  $\|A^T\| \geq c \cdot \rho^T$  pour une constante  $c > 0$  dépendant de  $P$  (équivalence des normes en dimension finie),  $\|A^T\| \rightarrow \infty$ . Si  $\rho < 1$ , chaque  $|\lambda_i^T| = |\lambda_i|^T \rightarrow 0$ , donc  $\Lambda^T \rightarrow 0$ , donc  $A^T = P\Lambda^T P^{-1} \rightarrow 0$ .  $\square$

##### Remarque

Ce résultat est démontré seulement dans le cas linéaire simplifié. Le cas général (avec non-linéarité  $f_\theta(h_{t-1}, x_t) = \sigma(Ah_{t-1} + Bx_t)$ ) suit le même principe, via la règle de la chaîne :  $\partial h_T / \partial h_0 = \prod_{t=1}^T \text{diag}(\sigma'(z_t)) A$ , un produit de  $T$  matrices. Si chaque facteur a une norme  $\leq \gamma < 1$ , le produit tend vers 0 exponentiellement (borne immédiate par sous-multiplicativité de la norme) ; le raisonnement symétrique s'applique à l'explosion. Nous admettons ce cas général sans le détailler davantage, la mécanique étant identique au cas linéaire déjà prouvé.

Ce fait explique une limite concrète des RNN : au-delà de quelques dizaines de pas de temps, le gradient devient inexploitable (trop petit pour apprendre, ou instable), ce qui limite la capacité du réseau à apprendre des dépendances entre éléments éloignés dans la séquence. Les LSTM [4] ont été conçus, empiriquement, pour atténuer ce phénomène via un mécanisme de portes ; ceci reste un remède partiel et empiriquement observé, pas une garantie théorique de préservation du gradient sur toute longueur de séquence.

### 2.6.5 Une seconde limite, indépendante du gradient

Même en ignorant ce problème, un RNN traite la séquence strictement séquentiellement :  $h_t$  ne peut être calculé qu'après  $h_{t-1}$ . Ceci empêche toute parallélisation du calcul le long de la séquence, un problème purement computationnel (temps d'entraînement) distinct du problème d'apprentissage (qualité du gradient) qu'on vient de discuter.

C'est précisément ces deux limites, prises ensemble (dépendances longues mal apprises, calcul non parallélisable), que le mécanisme d'attention va contourner.

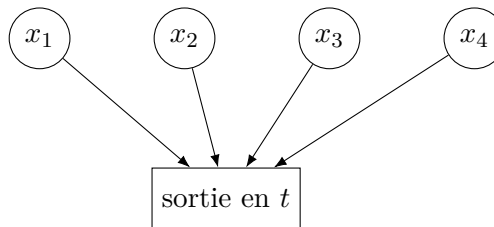
## 3 Attention et architecture Transformer

### 3.1 Motivation

Les deux limites dégagées en 2.6.4–2.6.5 sont : (i) un gradient qui s'atténue ou explose sur les dépendances longues, (ii) un calcul intrinsèquement séquentiel, non parallélisable. On voudrait une architecture où chaque position peut accéder directement à n'importe quelle autre, quelle que soit leur distance dans la séquence, et où le calcul pour toutes les positions s'effectue en parallèle.



Chaîne d'un RNN : le chemin entre  $h_0$  et  $h_3$  a pour longueur 3 (un pas par étape, longueur  $T$  en général).



Attention : chaque position accède directement à toutes les autres, chemin de longueur 1 quelle que soit  $T$ .

Le mécanisme d'attention, introduit ci-dessous, répond aux deux limites à la fois : accès direct (longueur de chemin  $O(1)$ ), et calcul parallélisable sur toutes les positions.

## 3.2 Le mécanisme d'attention

### 3.2.1 Projections

On part d'une séquence  $(x_1, \dots, x_T)$ ,  $x_t \in \mathbb{R}^d$  (issue du plongement, 2.5–2.6). On fixe trois matrices de poids  $W^Q, W^K \in \mathbb{R}^{d_k \times d}$ ,  $W^V \in \mathbb{R}^{d_v \times d}$ , et on pose, pour chaque  $t$  :

$$q_t = W^Q x_t, \quad k_t = W^K x_t, \quad v_t = W^V x_t,$$

appelés respectivement *requête* (query), *clé* (key), *valeur* (value) au temps  $t$ . En empilant ces vecteurs en lignes, on obtient les matrices  $Q, K \in \mathbb{R}^{T \times d_k}$ ,  $V \in \mathbb{R}^{T \times d_v}$ .

#### Nota

$QK^\top$ , à l'entrée  $(t, s)$ , vaut  $q_t \cdot k_s$  : le produit scalaire canonique sur  $\mathbb{R}^{d_k}$ . Un espace de Hilbert est un espace vectoriel muni d'un produit scalaire, complet pour la norme induite ;  $\mathbb{R}^{d_k}$ , muni de ce produit scalaire, en est un cas particulier où la complétude est automatique (toute suite de Cauchy dans un espace de dimension finie converge). On utilisera ce vocabulaire (orthogonalité, norme) sans abus dans la suite, tout en gardant à l'esprit que la dimension finie rend cette structure bien plus simple que la théorie générale des espaces de Hilbert.

### 3.2.2 La fonction softmax

#### Définition (Softmax)

Pour  $z \in \mathbb{R}^n$ ,  $\text{softmax}(z) \in \mathbb{R}^n$  est défini par  $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$ .

#### Remarque

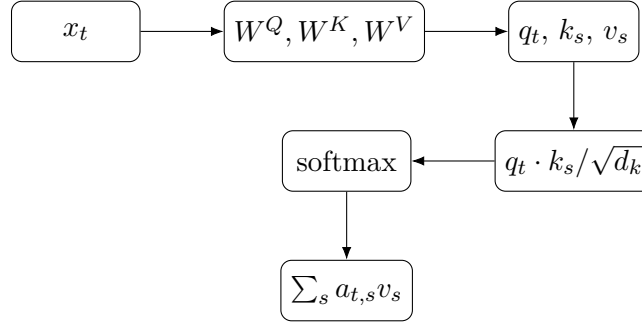
$\text{softmax}(z)_i > 0$  pour tout  $i$ , et  $\sum_i \text{softmax}(z)_i = 1$  :  $\text{softmax}(z)$  appartient toujours au simplexe ouvert  $\Delta^{n-1} = \{p \in \mathbb{R}_{>0}^n : \sum_i p_i = 1\}$ . C'est cette propriété, et elle seule, qui permettra le Lemme du barycentre plus bas.

### 3.2.3 Définition de l'attention

#### Définition (Attention)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

où le softmax s'applique ligne par ligne. Explicitement, la ligne  $t$  de la sortie est  $\sum_{s=1}^T a_{t,s} v_s$ , où  $a_t = \text{softmax}(q_t \cdot k_1 / \sqrt{d_k}, \dots, q_t \cdot k_T / \sqrt{d_k})$ .



Pipeline de calcul de la sortie d'attention à la position  $t$ .

### 3.2.4 Exemple numérique

Reprenons le vocabulaire jouet de 2.6.3 :  $V = \{\text{« le »}, \text{« chat »}, \text{« dort »}\}$ , avec les mêmes plongements  $x_1 = e(\text{« le »}) = (1, 0)$ ,  $x_2 = e(\text{« chat »}) = (0, 1)$ ,  $x_3 = e(\text{« dort »}) = (1, 1)$ . Prenons, pour simplifier,  $W^Q = W^K = W^V = I_2$  (matrice identité), donc  $Q = K = V = X$ , avec  $d_k = 2$ ,  $\sqrt{d_k} = \sqrt{2} \approx 1,414$ .

Les scores  $q_t \cdot k_s$  (matrice  $QK^\top$ ) sont :

$$QK^\top = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \quad \frac{QK^\top}{\sqrt{2}} \approx \begin{pmatrix} 0,707 & 0 & 0,707 \\ 0 & 0,707 & 0,707 \\ 0,707 & 0,707 & 1,414 \end{pmatrix}.$$

Pour la ligne  $t = 1$  (« le ») :  $\text{softmax}(0,707, 0, 0,707)$ . Avec  $e^{0,707} \approx 2,028$ ,  $e^0 = 1$ , la somme vaut  $\approx 5,056$ , d'où

$$a_1 \approx (0,401, 0,198, 0,401).$$

La sortie à la position 1 est alors

$$0,401 \cdot (1, 0) + 0,198 \cdot (0, 1) + 0,401 \cdot (1, 1) \approx (0,802, 0,599).$$

On observe que « le » distribue son attention presque également entre lui-même et « dort » (poids  $\approx 0,401$  chacun), et beaucoup moins sur « chat » ( $\approx 0,198$ ) : un effet direct du choix  $W^Q = W^K = I$ , qui rend le score  $q_1 \cdot k_s$  maximal lorsque  $x_s$  partage des coordonnées non nulles avec  $x_1$ . Le vecteur de sortie  $(0,802, 0,599)$  est, par construction, une combinaison convexe de  $v_1, v_2, v_3$  (Lemme du barycentre, ci-dessous) : on le vérifie directement, les coefficients  $(0,401, 0,198, 0,401)$  étant positifs et de somme 1.

### 3.2.5 Justification de la mise à l'échelle par $\sqrt{d_k}$

#### Proposition (Variance des scores)

Si les composantes de  $q_t$  et  $k_s$  sont i.i.d., centrées, de variance 1 (hypothèse d'initialisation idéalisée, pas garantie après entraînement), alors  $\text{Var}(q_t \cdot k_s) = d_k$ .

*Preuve.*  $q_t \cdot k_s = \sum_{i=1}^{d_k} q_{t,i} k_{s,i}$ , une somme de  $d_k$  termes indépendants (par indépendance des composantes). Donc  $\text{Var}(q_t \cdot k_s) = \sum_i \text{Var}(q_{t,i} k_{s,i})$ . Pour chaque terme,  $\mathbb{E}[q_{t,i} k_{s,i}] = \mathbb{E}[q_{t,i}] \mathbb{E}[k_{s,i}] = 0$  (indépendance, centrage), donc  $\text{Var}(q_{t,i} k_{s,i}) = \mathbb{E}[q_{t,i}^2 k_{s,i}^2] = \mathbb{E}[q_{t,i}^2] \mathbb{E}[k_{s,i}^2] = 1 \cdot 1 = 1$ . D'où  $\text{Var}(q_t \cdot k_s) = d_k$ .  $\square$

#### Remarque

Sans la division par  $\sqrt{d_k}$ , la variance des scores croît linéairement avec  $d_k$  ; pour  $d_k$  grand, les entrées de  $QK^\top$  prennent des valeurs de grande amplitude, ce qui pousse le softmax dans une région où son gradient est proche de 0 (saturation). Diviser par  $\sqrt{d_k}$  ramène la variance à 1, indépendamment de  $d_k$ . C'est un argument sous hypothèse idéalisée, qui motive le choix architectural sans le démontrer en toute généralité (l'hypothèse d'indépendance ne tient plus une fois le réseau entraîné).

### 3.2.6 Le lemme du barycentre

#### Lemme (Barycentre convexe)

Pour tout  $t$ , la ligne  $t$  de  $\text{Attention}(Q, K, V)$  appartient à l'enveloppe convexe de  $\{v_1, \dots, v_T\}$ .

*Preuve.* La ligne  $t$  est  $\sum_s a_{t,s} v_s$  avec  $a_{t,s} > 0$  et  $\sum_s a_{t,s} = 1$  (Remarque sur le softmax ci-dessus) : c'est exactement la définition d'une combinaison convexe des  $v_s$ .  $\square$

#### Remarque

Ce fait, trivial à démontrer, est structurant pour comprendre les limites du mécanisme : l'attention ne peut jamais produire un vecteur hors de l'enveloppe convexe de ce qu'elle regarde. Elle ne « génère » rien de nouveau au sens vectoriel, elle recombine. On y reviendra en Partie 6 (discussion des limites).

## 3.3 Multi-head attention

### 3.3.1 Motivation

Le lemme du barycentre (3.2.6) dit que la sortie d'une tête d'attention à la position  $t$  est *une seule* combinaison convexe des valeurs. Si la relation pertinente entre  $x_t$  et le reste de la séquence a plusieurs facettes différentes (par exemple, une relation syntaxique et une relation sémantique, simultanément), une seule distribution softmax ne peut pas les représenter toutes à la fois : elle doit choisir un unique compromis. L'idée est de calculer *plusieurs* attentions en parallèle, avec des projections différentes, puis de les recombinaer.



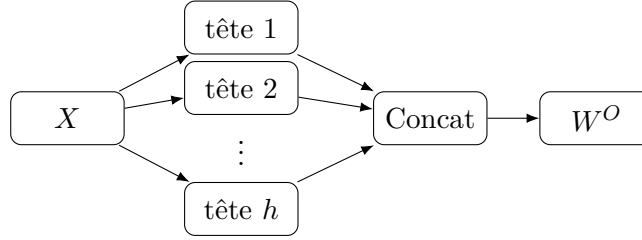
### 3.3.2 Définition

#### Définition (Multi-head attention)

Pour  $i = 1, \dots, h$ , on fixe des matrices de projection propres à la tête  $i$  :  $W_i^Q, W_i^K \in \mathbb{R}^{d_k \times d}$ ,  $W_i^V \in \mathbb{R}^{d_v \times d}$ , et on pose  $\text{head}_i = \text{Attention}(XW_i^{Q\top}, XW_i^{K\top}, XW_i^{V\top})$  (chaque tête recalcule ses propres  $Q, K, V$  à partir de la même entrée  $X$ , via 3.2.1). On concatène :

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O,$$

où  $W^O \in \mathbb{R}^{hd_v \times d}$ . En pratique,  $d_k = d_v = d/h$ , de sorte que le coût de calcul total reste comparable à une seule tête de dimension  $d$ .



Multi-head attention :  $h$  têtes calculées en parallèle à partir de  $X$ , concaténées puis reprojetées par  $W^O$ .

### 3.3.3 Un point à ne pas sur-interpréter

#### Remarque

On pourrait être tenté de dire que les  $h$  têtes explorent des sous-espaces orthogonaux de la représentation. Rien dans la définition ne le garantit : les  $W_i^Q, W_i^K, W_i^V$  sont des paramètres appris indépendamment par descente de gradient (2.3), sans contrainte d'orthogonalité imposée entre les têtes. Que certaines têtes se spécialisent dans des rôles syntaxiques distincts est un fait empirique observé après entraînement sur certains modèles, pas une propriété démontrable de l'architecture elle-même.

## 3.4 Nécessité d'un encodage positionnel

### 3.4.1 Un théorème d'équivariance

Rien dans la définition de l'attention (3.2.3) ne fait référence à la *position*  $t$  autrement que comme indice de sommation : deux séquences contenant les mêmes vecteurs dans un ordre différent produisent des sorties liées de façon totalement prévisible.

#### Théorème (Équivariance par permutation)

Soit  $\pi$  une permutation de  $\{1, \dots, T\}$  et  $P_\pi$  la matrice de permutation associée. Alors  $\text{Attention}(P_\pi X) = P_\pi \text{Attention}(X)$ .

*Preuve.* Comme  $Q = XW^Q$ ,  $(P_\pi X)W^Q = P_\pi(XW^Q) = P_\pi Q$ ; de même  $K' = P_\pi K$ ,  $V' = P_\pi V$ . Donc  $Q'K'^\top = P_\pi QK^\top P_\pi^\top$ . Le softmax s'applique ligne par ligne; la ligne  $i$  de  $Q'K'^\top$  est la

ligne  $\pi^{-1}(i)$  de  $QK^\top$ , avec ses colonnes elles-mêmes permutées par  $\pi$ , donc c'est exactement la même liste de nombres que la ligne  $\pi^{-1}(i)$  originale, réordonnée. Le softmax commute avec toute permutation des coordonnées d'un vecteur (c'est une fonction symétrique de ses arguments à réindexation près), donc  $\text{softmax}(Q'K'^\top/\sqrt{d_k}) = P_\pi A P_\pi^\top$ , où  $A = \text{softmax}(QK^\top/\sqrt{d_k})$ . La sortie est alors  $P_\pi A P_\pi^\top V' = P_\pi A P_\pi^\top P_\pi V = P_\pi A V = P_\pi \text{Attention}(X)$  (car  $P_\pi^\top P_\pi = I$ ).  $\square$

### Corollaire

Si  $\pi(t) = t$  (les autres positions sont permutées,  $t$  reste fixe), la ligne  $t$  de  $\text{Attention}(P_\pi X)$  est identique à la ligne  $t$  de  $\text{Attention}(X)$  : mélanger l'ordre du contexte autour de  $t$  ne change rien à ce que  $t$  « voit ». L'attention traite le contexte comme un *ensemble*, jamais comme une *suite* : aucune notion d'ordre n'existe sans information ajoutée explicitement.

### 3.4.2 L'encodage positionnel sinusoïdal

Il faut donc injecter, dans chaque  $x_t$ , une information qui dépend de  $t$ . La solution de Vaswani et al. [8] :  $x_t \leftarrow x_t + PE(t)$ , où  $PE(t) \in \mathbb{R}^d$  est défini par paires de coordonnées, pour  $i = 0, \dots, d/2 - 1$  :

$$PE(t)_{2i} = \sin(\omega_i t), \quad PE(t)_{2i+1} = \cos(\omega_i t), \quad \omega_i = 10000^{-2i/d}.$$

### Proposition (Position relative : une rotation fixe)

Pour une fréquence  $\omega$  fixée, il existe une matrice  $R_k \in \mathbb{R}^{2 \times 2}$ , dépendant de  $k$  mais pas de  $t$ , telle que

$$\begin{pmatrix} \sin(\omega(t+k)) \\ \cos(\omega(t+k)) \end{pmatrix} = R_k \begin{pmatrix} \sin(\omega t) \\ \cos(\omega t) \end{pmatrix}.$$

*Preuve.* Par les formules d'addition,  $\sin(\omega(t+k)) = \sin(\omega t) \cos(\omega k) + \cos(\omega t) \sin(\omega k)$  et  $\cos(\omega(t+k)) = \cos(\omega t) \cos(\omega k) - \sin(\omega t) \sin(\omega k)$ . C'est exactement  $R_k \begin{pmatrix} \sin \omega t \\ \cos \omega t \end{pmatrix}$  pour

$$R_k = \begin{pmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{pmatrix},$$

une matrice de rotation d'angle  $-\omega k$ , indépendante de  $t$ .  $\square$

### Remarque

Cette propriété, appliquée à chaque paire de coordonnées, dit que  $PE(t+k)$  s'obtient à partir de  $PE(t)$  par une transformation linéaire fixe (bloc-diagonale par rotations), la même quel que soit  $t$  : c'est cette structure, et non une simple envie de variété, qui motive le choix de fonctions sinusoïdales plutôt que, disons, un encodage appris arbitraire.

## 3.5 Architecture complète

### 3.5.1 Connexions résiduelles

On empile plusieurs blocs d'attention et de MLP (2.4) pour augmenter la capacité du réseau. Mais empiler des couches profondes réintroduit, par un mécanisme différent, un problème analogue à celui de 2.6.4 : le gradient doit traverser toutes les couches lors de la rétropropagation (2.3.4), et rien ne garantit qu'il ne s'atténue pas en chemin.

#### Définition (Connexion résiduelle)

Pour une sous-couche  $F_\theta$  (attention ou MLP), on remplace la sortie  $F_\theta(x)$  par  $x + F_\theta(x)$ .

#### Proposition (Chemin direct pour le gradient)

Pour  $y = x + F_\theta(x)$ ,  $\frac{\partial y}{\partial x} = I + \frac{\partial F_\theta}{\partial x}$ .

*Preuve.* Différentiation directe, linéarité de la dérivée d'une somme.  $\square$

#### Remarque

Quel que soit  $\partial F_\theta / \partial x$ , la matrice jacobienne contient toujours le terme  $I$  : même si  $\partial F_\theta / \partial x$  devient très petit (couche qui a appris à peu contribuer), le gradient dispose d'un chemin direct, non atténué, à travers l'identité. C'est un argument structurel sur la forme de la dérivée, pas une preuve que le gradient reste borné sur tout le réseau empilé (qui dépendrait de la composition de tous les blocs, pas d'un seul).

### 3.5.2 Normalisation

#### Définition (Layer normalization)

Pour  $x \in \mathbb{R}^d$ , notant  $\mu = \frac{1}{d} \sum_i x_i$ ,  $\sigma^2 = \frac{1}{d} \sum_i (x_i - \mu)^2$ ,

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta,$$

où  $\gamma, \beta \in \mathbb{R}^d$  sont des paramètres appris,  $\odot$  le produit terme à terme,  $\epsilon > 0$  une constante pour éviter la division par zéro.

#### Nota

Avant application de  $\gamma, \beta$ , le vecteur  $(x - \mu) / \sqrt{\sigma^2 + \epsilon}$  est, à  $\epsilon$  près, de norme  $\sqrt{d}$  constante (calcul immédiat :  $\sum_i \left( \frac{x_i - \mu}{\sigma} \right)^2 = \frac{1}{\sigma^2} \sum_i (x_i - \mu)^2 = \frac{d\sigma^2}{\sigma^2} = d$ ) : la normalisation projette (à un centrage près) tout vecteur sur une sphère de rayon  $\sqrt{d}$ , avant que  $\gamma, \beta$  ne redonnent au réseau la liberté d'ajuster l'échelle.

### 3.5.3 Réseau feedforward position-wise

#### Définition (FFN position-wise)

À chaque position  $t$ , indépendamment des autres, on applique le même MLP (2.4) à deux couches :  $\text{FFN}(x_t) = W_2 \sigma(W_1 x_t + b_1) + b_2$ .

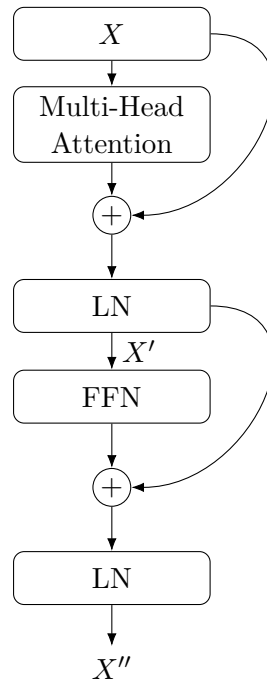
« Position-wise » signifie que ce même MLP, avec les mêmes poids, est appliqué séparément à chaque ligne de la séquence, c'est le pendant, au niveau d'une position isolée, de ce que l'attention fait au niveau de la séquence entière : l'attention mélange l'information *entre* positions, le FFN transforme l'information à *l'intérieur* de chaque position.

### 3.5.4 Un bloc complet

#### Définition (Bloc encodeur)

Un bloc encodeur transforme  $X$  en :

$$X' = \text{LN}(X + \text{MultiHead}(X)), \quad X'' = \text{LN}(X' + \text{FFN}(X')).$$



Un bloc encodeur : deux sous-couches (attention, FFN), chacune entourée d'une connexion résiduelle (boucles à droite) et suivie d'une normalisation.

Un encodeur complet empile  $N$  blocs identiques dans leur structure (mais avec des poids propres à chaque bloc) :  $X^{(0)} = X$ ,  $X^{(i)} = \text{Bloc}(X^{(i-1)})$ .

### Remarque

Le décodeur reprend la même structure, avec deux différences que nous ne détaillons pas ici (hors du périmètre de cet article, centré sur l’analyse d’image, qui n’utilise que l’encodeur) : un masquage du score d’attention empêchant une position d’accéder aux positions futures, et une seconde attention croisée entre les représentations du décodeur et celles de l’encodeur.

## Références

- [1] Haïm Brezis. *Functional Analysis, Sobolev Spaces and Partial Differential Equations*. Springer, 2010.
- [2] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4) :303–314, 1989.
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8) :1735–1780, 1997.
- [5] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. MIT Press, 2nd edition, 2018.
- [6] Walter Rudin. *Real and Complex Analysis*. McGraw-Hill, 3rd edition, 1987.
- [7] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323 :533–536, 1986.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008. Curran Associates, Inc., 2017.